

Centro Universitário Filadélfia
Polo Salinópolis
Ciência de Dados e Inteligência Artificial

Aula 11

Empacotamento e Distribuição
Fundamentos de Propriedade Intelectual Para a Transferência de
Tecnologia de Software

Thiago Rafael da Silva Moura
trsmoura@yahoo.com.br

28 de novembro de 2025

CAPÍTULO 11

EMPACOTAMENTO & DISTRIBUIÇÃO

Transformando Scripts em Bibliotecas Reutilizáveis

[OBS: Complementando o Cap 10 [Book] que aborda Packaging]

01/40

Recapitulando a Jornada

No Capítulo 10, aprendemos a empacotar **Aplicações** (APIs) em Containers Docker.

No Capítulo 11, vamos dar um passo atrás e aprender a empacotar **Bibliotecas**.

Por que? Para que você possa compartilhar suas funções de Data Science entre múltiplos projetos (e containers) usando um simples `pip install`.

Blueprint da Aula



1. Módulos & Pacotes

Estrutura básica, `__init__.py` e importações.



2. Gestão de Dependências

Poetry vs Pip, e o arquivo `pyproject.toml`.



3. Publicação

Builds, Wheels e upload para o PyPI.

O Problema do "Copy-Paste"

Você cria uma função incrível de limpeza de dados no Projeto A.

Agora você precisa dela no Projeto B.

Solução Ruim: Copiar o arquivo `utils.py`.

Problema: Se você corrigir um bug no Projeto A, o Projeto B continua quebrado. A "Verdade" se fragmenta.

600 × 400

A Solução: Bibliotecas

Extraia o código comum para um **Pacote Python** separado.

```
# No Projeto A pip install minha-lib-ds from minha_lib import limpar_dados # No Projeto B pip install minha-lib-d
```

Uma única fonte de verdade. Atualize a lib, atualize todos os projetos.

Módulo vs Pacote

Módulo

É apenas um arquivo único com extensão `.py`.

```
meu_script.py
```

Simple, mas difícil de escalar.

Pacote

É uma pasta contendo um arquivo especial `__init__.py` e múltiplos módulos.

```
meu_pacote/  
├── __init__.py  
├── utils.py  
└── models.py
```

O arquivo `\_\_init\_\_.py`

Ele diz ao Python: "Esta pasta é um pacote importável".

Também é usado para expor funções para o mundo externo, controlando o que o usuário vê.

```
# meu_pacote/__init__.py # Traz a função 'limpar' de 'utils.py' para a raiz do pacote from .utils import limpar #
```

Estrutura de Diretórios (Src Layout)

O padrão moderno recomenda usar uma pasta `src` para evitar conflitos de importação durante testes.

```
meu-projeto/  
├── pyproject.toml  
├── README.md  
├── src/  
│   ├── minha_lib/  
│   │   ├── __init__.py  
│   │   └── core.py  
│   ├── tests/  
│   └── test_core.py
```

Isso garante que você teste o pacote **instalado**, não apenas os arquivos locais.

Imports Relativos

Dentro do seu pacote, evite hardcodar o nome do pacote. Use o ponto ..

```
# Em src/minha_lib/models.py # RUIM: Quebra se você renomear a pasta 'minha_lib' from minha_lib.utils import help
```



PARTE 2: DEPENDÊNCIAS

O inferno do requirements.txt e a salvação do Poetry

O problema do requirements.txt

```
pandas=1.5.0 numpy=1.23.0 scikit-learn=1.1.0
```

1. **Sem separação:** Mistura dependências diretas (o que você usa) com sub-dependências (o que o Pandas usa).
2. **Conflitos:** O pip antigo não resolvia bem versões incompatíveis.
3. **Sem distinção:** Mistura ferramentas de dev (pytest, black) com produção.

Conheça o Poetry

Poetry é uma ferramenta moderna que substitui pip, venv, setuptools e twine.

Ele gerencia dependências, cria ambientes virtuais e empacota seu código. Tudo em um.



pyproject.toml: O Novo Padrão

Introduzido na PEP 518, é o arquivo de configuração universal para projetos Python.

```
[tool.poetry] name = "minha-lib-ds" version = "0.1.0" description = "Ferramentas de DS da Empresa X" authors
```

Iniciando com Poetry

Para começar um projeto novo:

```
$ poetry new minha-lib
```

Isso cria a estrutura de pastas perfeita (src, tests, pyproject.toml) automaticamente.

Para adicionar uma biblioteca:

```
$ poetry add pandas
```

O Arquivo de Lock (poetry.lock)

O `pyproject.toml` diz: "Quero Pandas versão 2 ou superior".

O `poetry.lock` diz: "Instalei exatamente Pandas 2.0.3, com Numpy 1.24.1, etc.".

Benefício: Garante que todos no time (e na produção) tenham **bit-a-bit** o mesmo ambiente.

Ambientes Virtuais Automáticos

Você não precisa mais fazer `python -m venv venv`.

O Poetry cria e gerencia o ambiente virtual para você nos bastidores.

```
# Rodar um script dentro do env do projeto $ poetry run python main.py # Entrar no shell do env $ poetry shell
```

Versionamento Semântico (SemVer)

Como numerar suas versões? Use o padrão MAJOR.MINOR.PATCH (ex: 1.2.4).

MAJOR (1.x)

Mudanças que quebram compatibilidade (Breaking Changes).

MINOR (x.2)

Novas funcionalidades, mas compatível com anterior.

PATCH (x.x.4)

Correções de bugs apenas.

O acento circunflexo (^)

No `pyproject.toml`, você vê `pandas = "^2.0.0"`.

Isso significa: "Aceito qualquer versão que não quebre meu código".

:: Aceita: 2.1.0, 2.9.9

:: Rejeita: 3.0.0 (Pois MAJOR muda, risco de quebra).

O Poetry resolve essa matemática para você.



PARTE 3: PUBLICAÇÃO

Compartilhando com o mundo (ou com a empresa)

Artefatos de Build

Para distribuir, transformamos o código em dois arquivos:

sdist (.tar.gz)

Source Distribution. Contém o código fonte puro. O usuário precisa compilar se tiver C/C++.

wheel (.whl)

Built Distribution. Pré-compilado e pronto para usar. Instalação muito mais rápida.

Construindo o Pacote

Com Poetry, é um comando:

```
$ poetry build Building minha-lib-ds (0.1.0) - Building sdist - Building wheel
```

Isso cria uma pasta `dist/` com os arquivos `.whl` e `.tar.gz`.

PyPI (Python Package Index)

O repositório oficial onde vivem Pandas, Numpy, etc.

Qualquer um pode publicar lá. É público e gratuito.

Para empresas, usamos **PyPI Privado** (Artifactory, AWS CodeArtifact).

600 × 400

Publicando

Primeiro, configure suas credenciais (Token do PyPI):

```
$ poetry config pypi-token.pypi meu-token-secreto
```

Depois, publique:

```
$ poetry publish
```

Pronto! Agora o mundo todo pode fazer `pip install minha-lib-ds`.

TestPyPI

Não suje o PyPI oficial com testes!

Use o **TestPyPI**, uma cópia exata do PyPI feita para você praticar upload sem medo.

```
$ poetry config repositories.testpypi https://test.pypi.org/legacy/ $ poetry publish -r testpypi
```



PARTE 4: DOCUMENTAÇÃO

Se não está documentado, não existe

Docstrings

Documentação que vive dentro do código.

Use o estilo **Google** ou **NumPy** para Data Science (mais legível).

```
def media(lista): """Calcula a média. Args: lista (list): Lista de números. Returns: float: A média aritmética. """
```

MkDocs: Sites de Documentação

Transforma seus arquivos Markdown (README.md, docs/*.md) em um site estático lindo.

Muito mais simples que o Sphinx.

Tema popular: `mkdocs-material`.

600 × 400

Auto-Doc (mkdocstrings)

Você não quer escrever a documentação duas vezes (no código e no site).

O plugin `mkdocstrings` lê suas Docstrings do código Python e as insere automaticamente no site do MkDocs.

Mudou o código? O site atualiza sozinho.



DICAS PRÁTICAS

Evitando armadilhas comuns

Arquivos de Dados (MANIFEST.in)

Por padrão, o Python só empacota arquivos `.py`.

Se sua lib precisa de um CSV de config ou um modelo `.pkl` pequeno, você precisa incluí-los explicitamente.

No Poetry, adicione ao `include` no `pyproject.toml`.

Entry Points (CLI)

Quer que seu pacote funcione como um comando de terminal? (Como o próprio poetry ou black?)

```
[tool.poetry.scripts] meu-app = "meu_pacote.main:start"
```

Após instalar, o usuário pode digitar `$ meu-app` no terminal.

Executando o Pacote (`\_\_main\_\_.py`)

Se você adicionar um arquivo `__main__.py` na raiz do pacote, você pode executá-lo diretamente com:

```
$ python -m meu_pacote
```

Isso é muito comum para ferramentas e servidores.

Importações Cíclicas

Erro: Módulo A importa B, e B importa A.

Isso trava o Python.

Solução: Refatore! Mova a lógica comum para um terceiro Módulo C. Ou use importação dentro da função (runtime import).

Templates com Cookiecutter

Não configure tudo do zero toda vez.

Use **Cookiecutter** para gerar estruturas de projeto prontas (Data Science, Libs, APIs).

```
$ cookiecutter https://github.com/drivendata/cookiecutter-data-science
```

Monorepo vs Multirepo

Multirepo

Cada lib em um repositório Git. Bom para isolamento, ruim para refatoração em massa.

Monorepo

Todas as libs da empresa no mesmo Git. Fácil de compartilhar código, mas o CI fica complexo.

Estratégia Corporativa

Empresas não publicam segredos no PyPI público.

Configure um índice privado (AWS CodeArtifact, Azure Artifacts).

Configure o `pip/poetry` para buscar primeiro no seu índice privado, depois no público.

600 × 400

Conectando os Pontos

Cap 10 (Book) - Packaging: Criamos a lib `minha-lib-ds` versão 1.0.0.

Cap 11 (Book) - Deployment: Criamos uma API (FastAPI) que importa essa lib:

```
# requirements.txt da API fastapi minha-lib-ds==1.0.0
```

Isso é Engenharia de Software real: Componentização.

Recursos para Estudo

- :: **Site:** python-poetry.org
- :: **Site:** packaging.python.org (Guia Oficial)
- :: **Ferramenta:** mkdocs-material.squidfunk.com
- :: **Artigo:** "Hypermodern Python" (cjolowicz)



DÚVIDAS?

Sobre Poetry, PyPI ou Dependências?